

METU Computer Engineering

CEng 111 – Fall 2026
Week 5a

Credit: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.

Previously on CENG111!

Choosing a PL

- Ex: Moving soil with a shovel and a grader



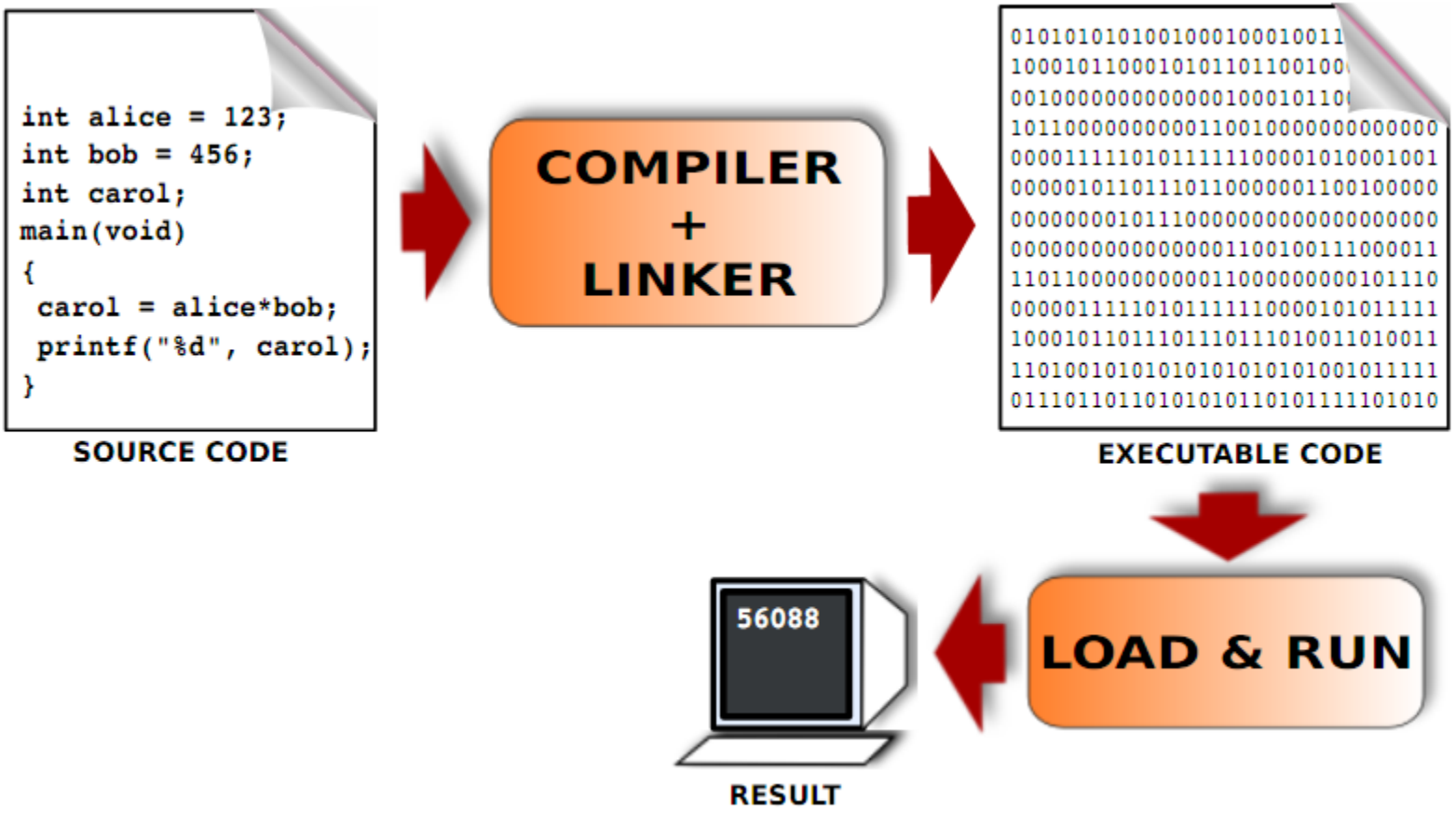


How are languages implemented

Previously on CENG111!

METU Computer Engineering

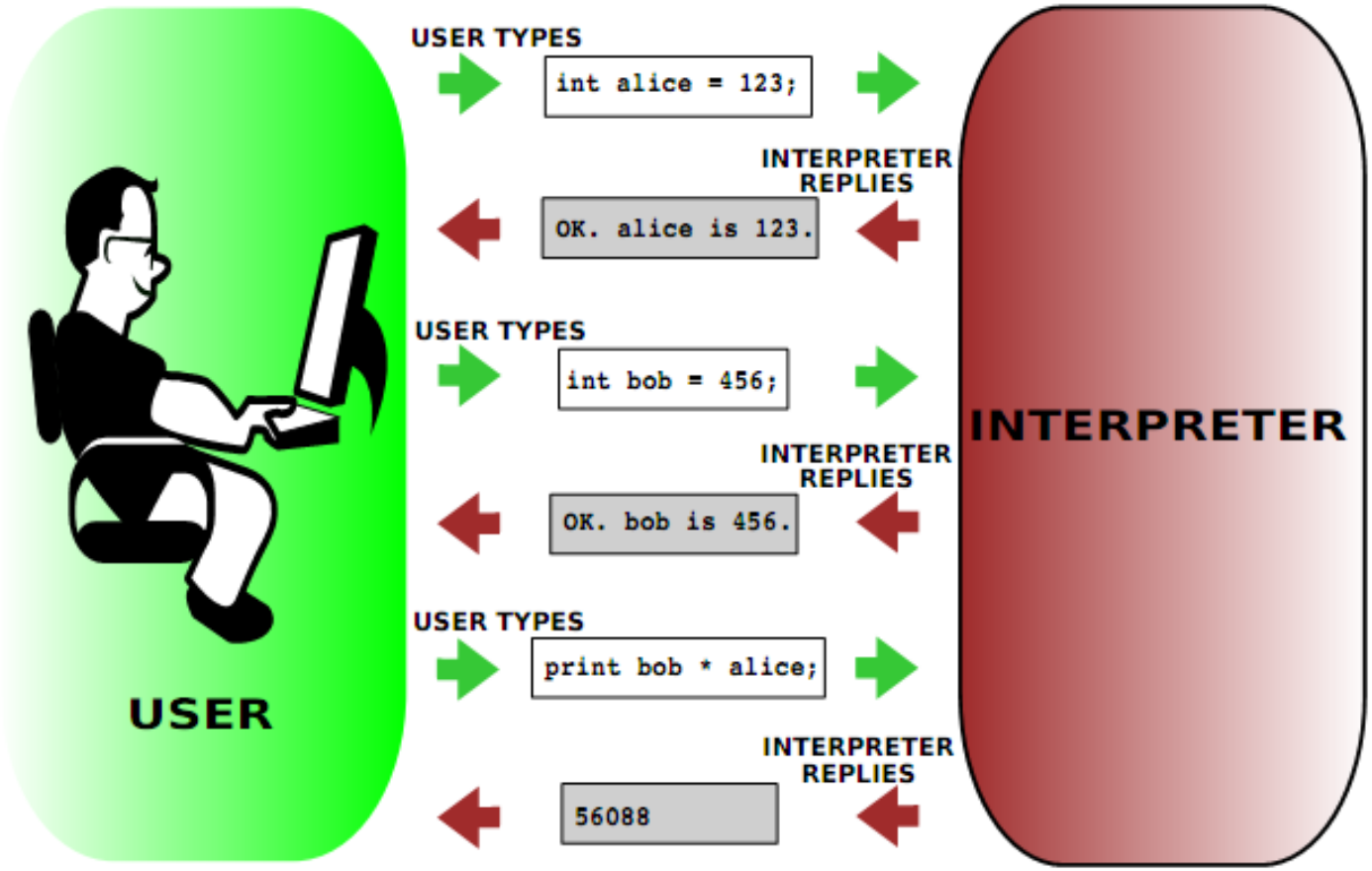
COMPILATIVE APPROACH



How are languages implemented

Previously on CENG111!

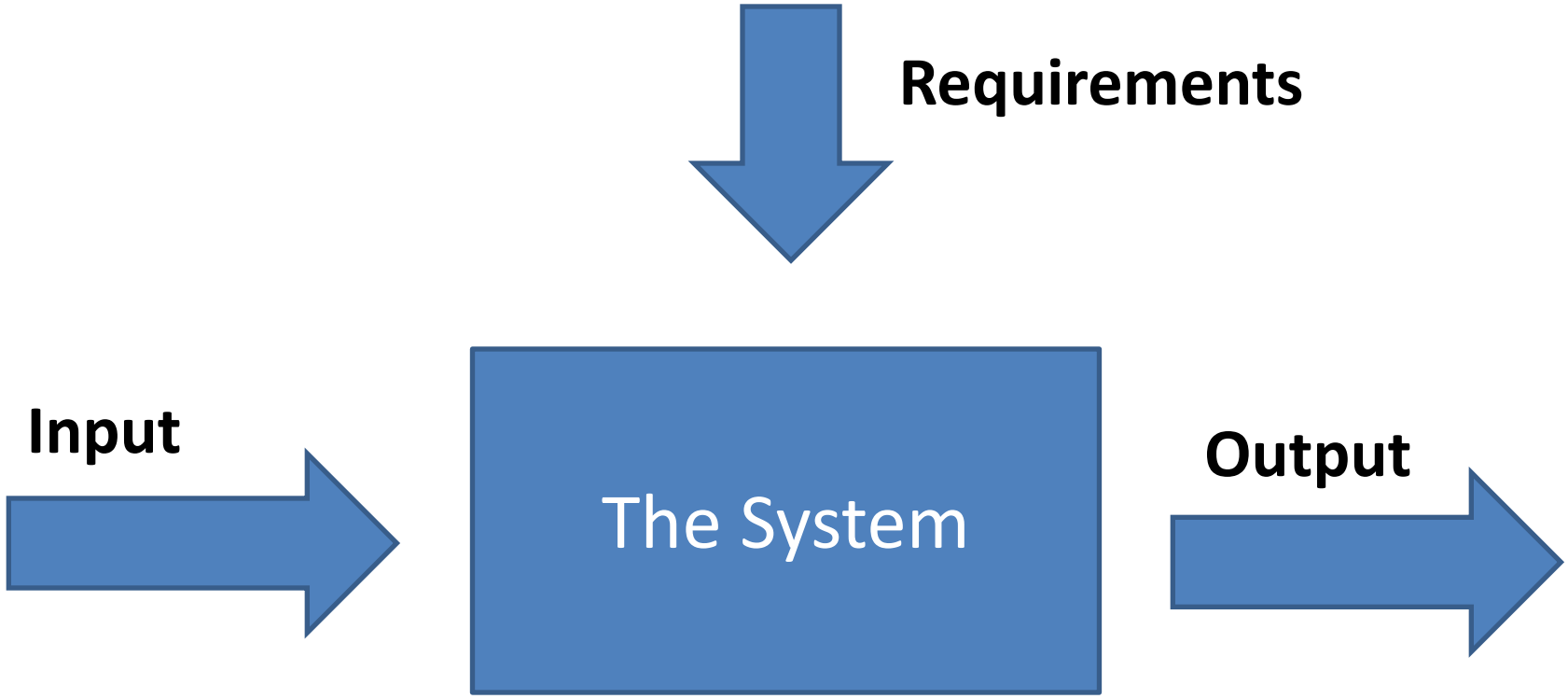
INTERPRETIVE APPROACH



Previously on CENG111!

Testing

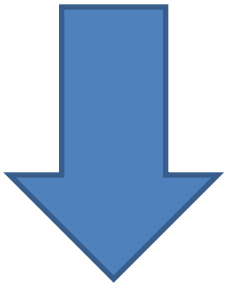
- Black-box Testing



Testing

Previously on CENG111!

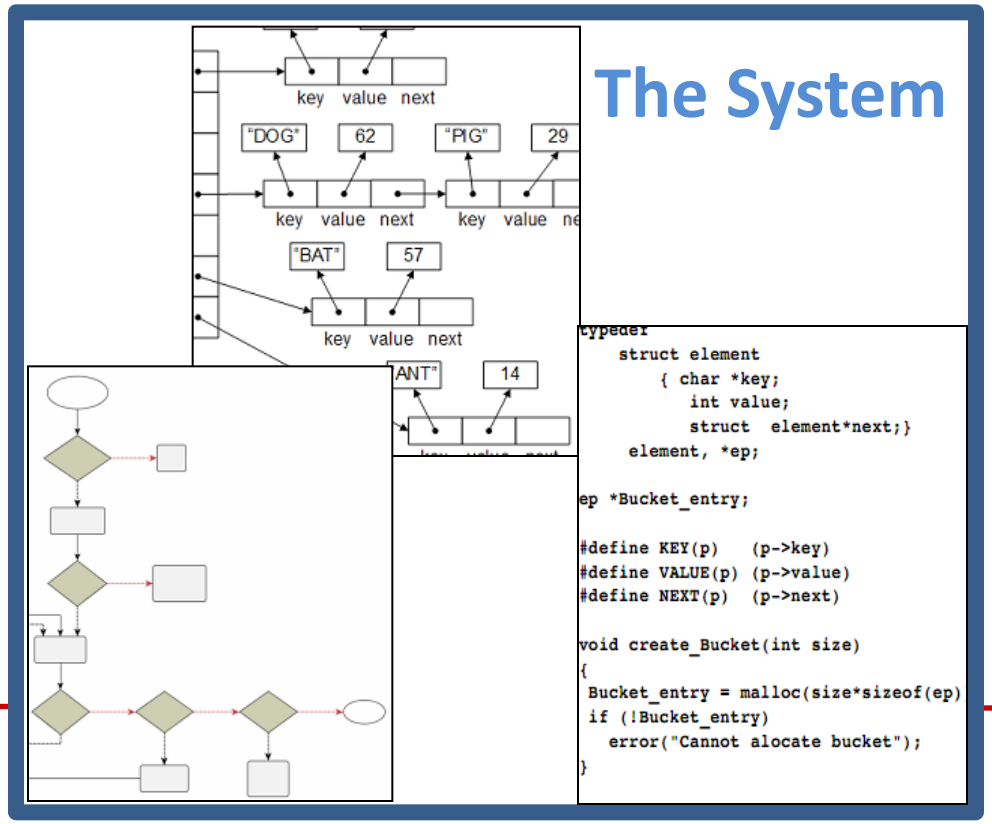
- White-box Testing



Requirements

Input





Output





Previously on CENG111!

Bugs, Errors

- Syntax Errors

  Area = 3.1415 * R * R

  Area = 3.1415 x R x R

- Run-time Errors

```
>>> def SqrtDelta(a,b,c):
>>>       return sqrt(b*b - 4*a*c)
>>>
>>> print SqrtDelta(1,3,1)
2.236067977497898
>>> print SqrtDelta(1,1,1)
ValueError: math domain error
```

Previously on CENG111!

Bugs, Errors

- Logical Errors

$$root_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$



```
>>> root1 = (- b + sqrt(b*b - 4*a*c)) / 2*a
```

- Design Errors

$$x^3 + ax^2 + bx + c = 0$$

$$root_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$



Previously on CENG111!




```
skalkan@divan:~$ python
Python 2.5.2 (r252:60911, Jan 24 2010, 17:44:40)
[GCC 4.3.2] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```



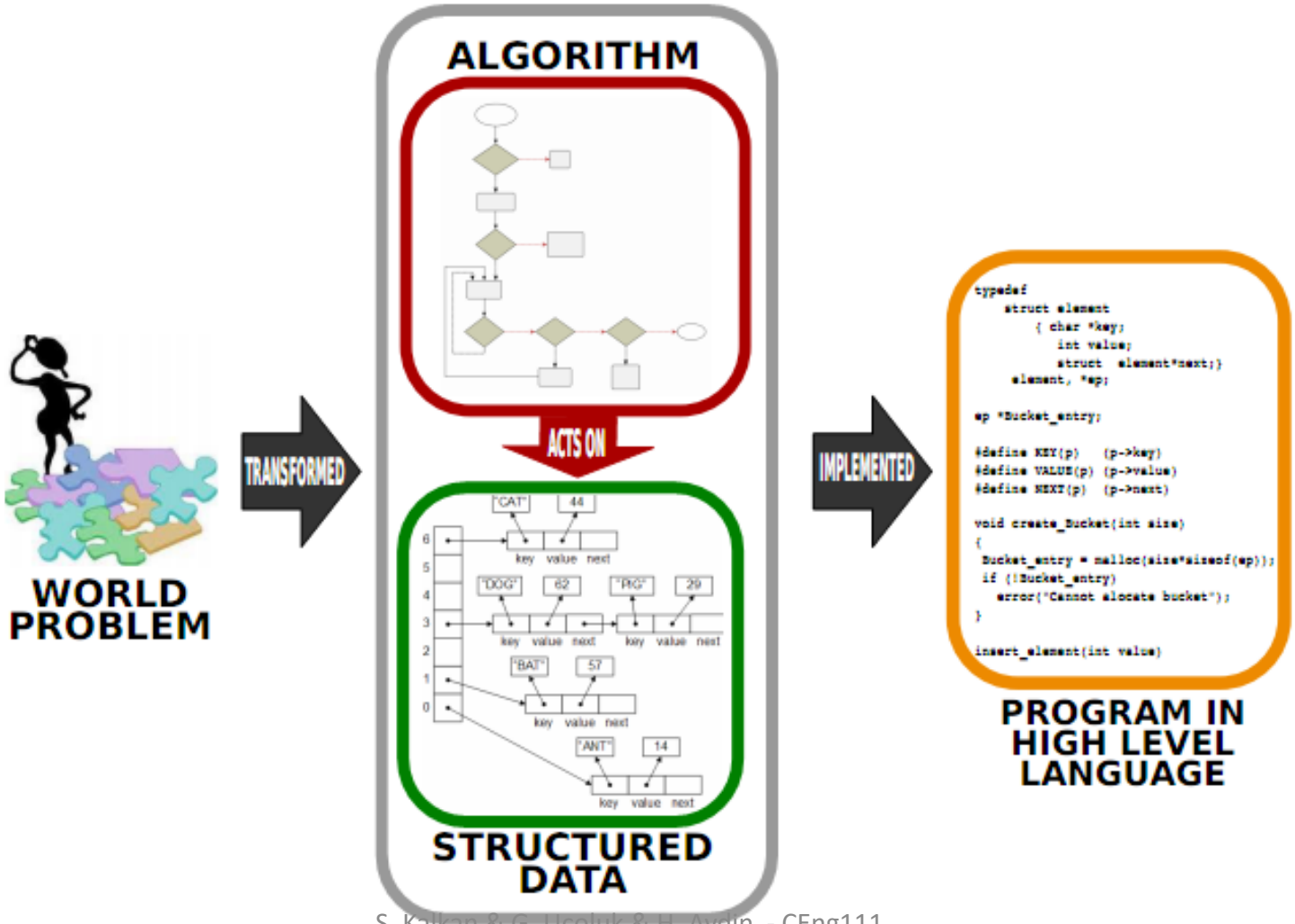
Today

- Binary representation
  - Integers
  - Real Numbers
  - Characters
  - Other types of information

Administrative Notes

- Midterm Exam
  - Date: 20 November, 18:00
- Lab Exams
  - Lab Exam 1: 8 November, Sunday, 13:00
  - Lab Exam 2: 6 December, Sunday, 13:00
  - Lab Exam 3: 20 December, Sunday, 13:00
  - Lab Exam 4: 3 January, Sunday, 13:00

Design of a solution

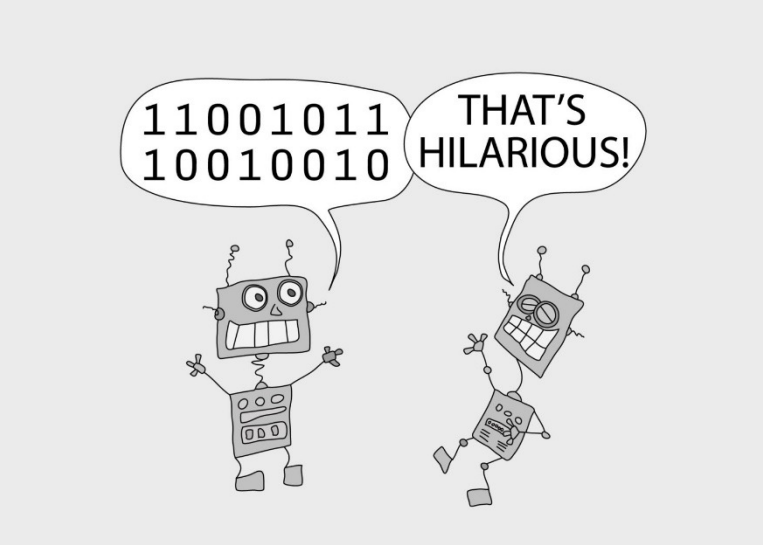


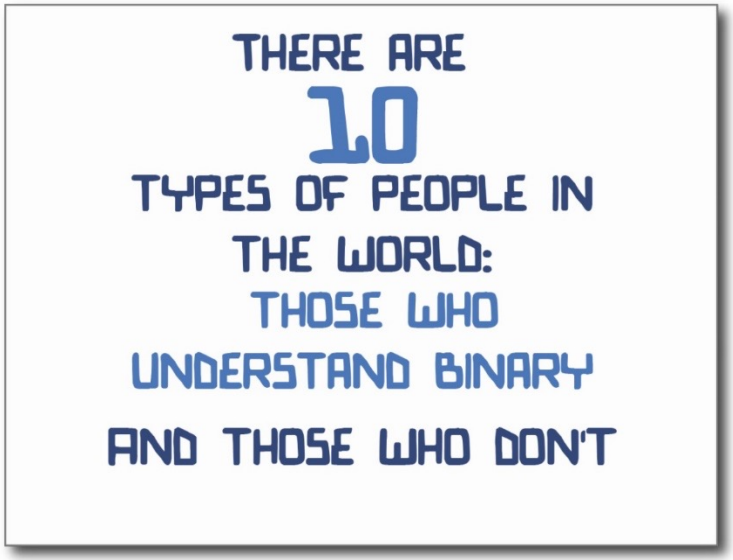
What is data?

- **Data**: Information to be processed to solve a problem.
- Identify the data for the following example problems:
  - Find all wheat growing areas in a terrestrial satellite image.
  - Given the homework, lab and examination grades of a class, calculate the letter grades.
  - Alter the amplitude of a sound recording for various frequencies.
  - Extrapolate China's population for the year 2040 based on the change in the population growth rate up to this time.
  - Compute the launch date and the trajectory for a space probe so that it will pass by the outermost planets in the closest proximity.
  - Compute the layout of the internals of a CPU so that the total wiring distance is minimized.
  - Find the cheapest flight plan from A to B, for given intervals for arrival and departure dates.
  - Simulate a war between two land forces, given (i) the attack and the defense plans, (ii) the inventories and (iii) other attributes of both forces.

What is data?

- CPU can only understand two types of data:
  - Integers,
  - Floating points.
- The following are not directly understandable by a CPU:
  - Characters (‘a’, ‘A’, ‘2’, …)
  - Strings (“apple”, “banana”, …)
  - Complex Numbers
  - Matrices
  - Vectors
- But, programming languages can implement these data types.





BINARY REPRESENTATION



Data Representation

- Based on 1s and 0s
  - So, everything is represented as a set of binary numbers
- We will now see how we can represent:
  - Integers: 3, 1234435, -12945 etc.
  - Floating point numbers: 4.5, 124.3458, -1334.234 etc.
  - Characters: /, &, +, -, A, a, ^, 1, etc.
  - …

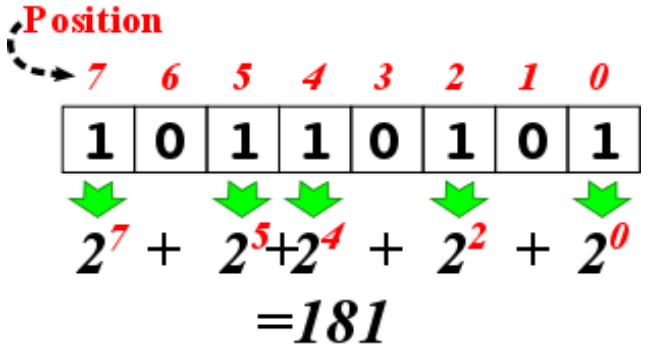
Binary Representation of Numeric Information

- Decimal numbering system
  - Base-10
  - Each position is a power of 10

    $3052 = 3 \times 10^3 + 0 \times 10^2 + 5 \times 10^1 + 2 \times 10^0$

- Binary numbering system
  - Base-2
  - Uses ones and zeros
  - Each position is a power of 2

    $1101 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$



Decimal-to-binary Conversion

	Dividend		Divisor		Quotient	Remainder
Step 1	19	÷	2	=	9	1
Step 2	9	÷	2	=	4	1
Step 3	4	÷	2	=	2	0
Step 4	2	÷	2	=	1	0
Step 5	1	÷	2	=	0	1

Continue until quotient is zero

The result:

1	0	0	1	1

Binary Representation of Numeric Information (continued)

- Representing integers (negative and positive)
  - Decimal integers are converted to binary integers
  - Question: given k bits, what is the value of the largest integer that can be represented?
  - $2^k - 1$
    - Ex: given 4 bits, the largest is $2^4-1 = 15$
- Signed integers must also represent the sign (positive or negative) - ***Sign/Magnitude notation***

Binary Representation of Numeric Information (continued)

- Sign/magnitude notation

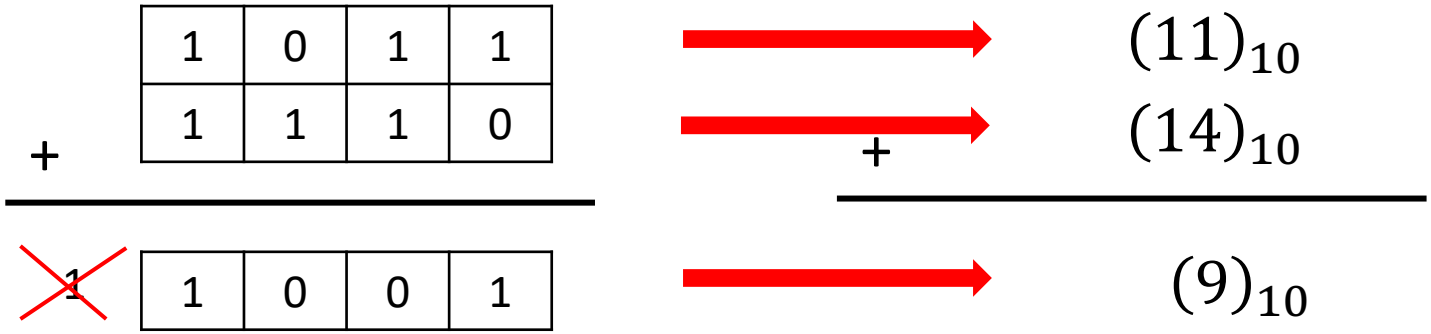
  1 101 = - 5

  0 101 = + 5

- Problems:
  - Two different representations for 0:
    - 1 000 = - 0
    - 0 000 = +0
  - Addition & subtraction require a watch for the sign! Otherwise, you get incorrect results:
    - 0 010 (+2) + 1 010 (-2) = 1 100 (-4)

Arithmetic in Computers is Modular

Let's add two numbers in binary
(Assume that there is no sign bit)



In other words:

- Numbers larger than or equal to 16 (2^4) are discarded in a 4-bit representation.
- Therefore, $11 + 14$ yields 9 in this 4-bit representation.
- This is actually modular arithmetic:

$$11 + 14 \; mod \; 16 \equiv 9 \; mod \; 16$$

For more info about modular arithmetic: https://brilliant.org/wiki/modular-arithmetic/

Binary Representation of Numeric Information (continued)

- Two's complement instead of sign-magnitude representation
  - Positive numbers have a leading 0.
    - 5 => 0101
  - The representation for negative numbers is found by subtracting the absolute value from 2^N for an N-bit system:
    - -5 => $2^4 - 5 = 16 - 5 = (11)_{10} => (1011)_2$
- Advantages:
  - 0 has a single representation: +0 = 0000, -0 = 0000
  - Arithmetic works fine without checking the sign bit:
    - 1011 (-5) + 0110 (6) = 0001 (1)
    - 1011 (-5) + 0011 (3) = 1110 (-2)

Binary Representation of Numeric Information (continued)

- Shortcut to convert from "two's complement" :
  - If the leading bit is zero, no need to convert.
  - If the leading bit is one, invert the number and add 1.
- What is our range?
  - With 2's complement we can represent numbers from -2^{N-1} to $2^{N-1} - 1$ using N bits.
  - 8 bits: -128 to +127.
  - 16 bits: -32,768 to +32,767.
  - 32 bits: -2,147,483,648 to +2,147,483,647.

Binary Number	Decimal Value	Value in Two's Complement
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

Binary Representation of Numeric Information (continued)

- Example:
  - We want to compute: 7 – 3
  - 7 => 0111
  - -3 => -(0011) => (1100)+1 => (1101)
- 7 – 3 =

```
  0111
+ 1101
-----------
 00100 => 4
```

So, addition and subtraction operations are simpler in the Two's Complement representation

METU Computer Engineering



Binary Representation of Numeric Information (continued)

- Due to its advantages, two's complement is the most common way to represent integers on computers.



Why does Two's Complement work?

- Inversion and addition of a 1-bit correspond effectively to subtraction from 0 – i.e., negative of a number.
- Negative of a binary number X: $(00\ldots00)_2 - (X)_2$
- Note that $(00\ldots00)_2 = (11\ldots11)_2 + (1)_2$
- In other words:
  - $(00\ldots00)_2 - (X)_2 = \underline{(11\ldots11)_2 - (X)_2} + (1)_2$.

(i.e., how we find two's complement)

Inversion

Why does Two's Complement work?

- A smart trick used in mechanical calculators
  - To subtract b from a, invert b and add that to a. Then discard the most significant digit.



http://en.wikipedia.org/wiki/Method_of_complements

METU Computer Engineering



METU Computer Engineering

Example range for 4 bits

- We divide the range of unsigned numbers using N bits into two halves:
- 0 to $2^{N-1} - 1$ => Nonnegative numbers
- 2^{N-1} to $2^{N} - 1$ => Negative numbers (-2^{N-1} to -1)

Binary Number	Decimal Value	Value in Two's Complement
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

Binary Clock in St. Gallen, Switzerland



https://en.wikipedia.org/wiki/St._Gallen_railway_station

Binary Representation of Real Numbers

Conversion of the digits after the dot into binary:

- 1st Way:
  - 0.375 → 0x½ + 1x¼ + 1x1/8 → 011
- 2nd Way:

	Fraction		Multiplier		Whole		Fraction
Step 1	0.375	×	2	=	0	.	75
Step 2	0.75	×	2	=	1	.	5
Step 3	0.5	×	2	=	1	.	0

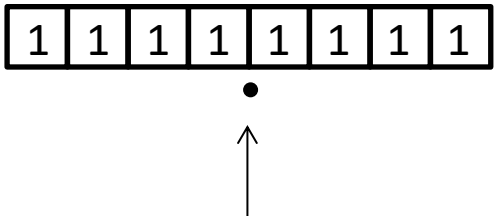
The result:

.	0	1	1

Continue until fraction is zero

Binary Representation of Real Numbers

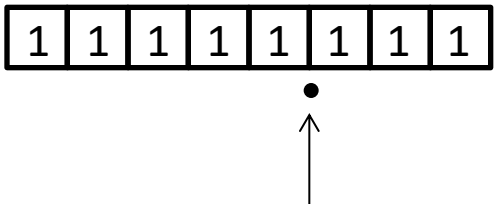
- Approach 1: Use fixed-point
  - Similar to integers, except that there is a decimal point.
  - E.g.: using 8 bits:



$= 1\times2^3 + 1\times2^2 + 1\times2^1 + 1\times2^0 + 1\times2^{-1} + 1\times2^{-2} + 1\times2^{-3} + 1\times2^{-4}$
$= 15.9375$

Binary Representation of Real Numbers

- Location of the decimal point changes the value of the number.
  - E.g.: using 8 bits:



$= 1\times2^4 + 1\times2^3 + 1\times2^2 + 1\times2^1 + 1\times2^0$
$1\times2^{-1} + 1\times2^{-2} + 1\times2^{-3} = 31.875$



Binary Representation of Real Numbers

- Problems with fixed-point:
  - Limited in the maximum and minimum values that can be represented.
  - For instance, using 32-bits, reserving 1-bit for the sign and putting the decimal point after 16 bits from the right, the maximum positive value that can be stored is slightly less than 2^{15}.
  - Allowing larger values gives away from the precision (the decimal part).

Binary Representation of Real Numbers

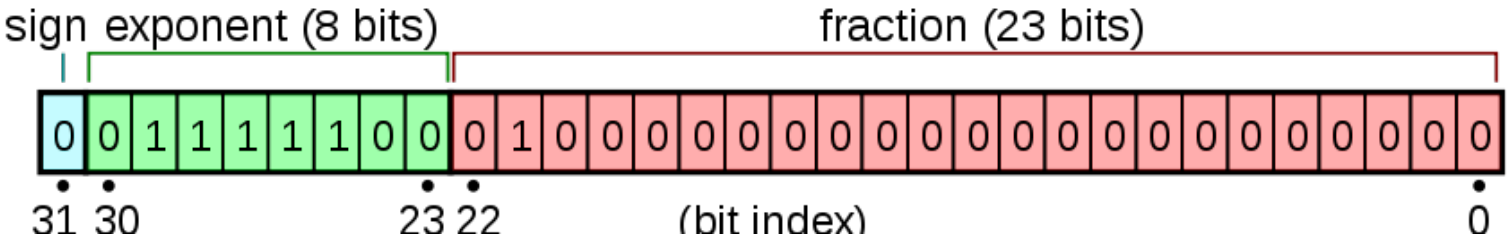
- Solution: Use scientific notation: a x 2^b *(or* $\pm M \times B^{\pm E}$*)*
  - Example: 5.75
    - 5 → 101
    - 0.75 → ½ + ¼ → $2^{-1}+2^{-2}$ → $(0.11)_2$
    - 5.75 → $(101.11)_2 \times 2^0$
- Number is then normalized so that the first significant digit is immediately to the left of the binary point
  - Example: 1.0111×2^2
- We take and store the mantissa and the exponent.



Binary Representation of Real Numbers

- This needs some standardization for:
  - where to put the decimal point
  - how to represent negative numbers
  - how to represent numbers less than 1

IEEE 32bit Floating-Point Number Representation



= 0.15625

$$= (-1)^{\text{sign}}(1.b_{-1}b_{-2}...b_{-23})_2 \times 2^{e-127}$$

- M x 2^E $(2-2^{-23})\times 2^{127}$
- Exponent (E): 8 bits
  - Add 127 to the exponent value before storing it
  - **E can be 0 to 255 with 127 representing the real number zero (0).**
- Fraction (M - Mantissa): 23 bits
- $2^{128} = 3.4028237 \times 10^{38}$

IEEE 32bit Floating-Point Number Representation

- Example: 12.375
- The digits before the dot:
  - $(12)_{10} \rightarrow (1100)_2$
- The digits after the dot:
  - 1st Way: 0.375 → 0x½ + 1x¼ + 1x1/8 → 011
  - 2nd Way: Multiply by 2 and get the integer part until 0:
    - 0.375 x 2 = 0.750 = 0 + 0.750
    - 0.750 x 2 = 1.50 = 1 + 0.50
    - 0.50 x 2 = 1.0 = 1 + 0.0
- $(12.375)_{10} = (1100.011)_2$
- NORMALIZE (move the point): $(1100.011)_2 = (1.100011)_2 x 2^3$
- Exponent: 3, adding 127 to it, we get 1000 0010
- Fraction: 100011
- Then our number is: 0 10000010 10001100000000000000000



Why add bias to the exponent?

- It helps in comparing the exponents of the same-sign real-numbers without looking out for the sign of the exponent.

Binary Number	Decimal Value	Value in Two's Complement	Value with bias 7
0000	0	0	-7
0001	1	1	-6
0010	2	2	-5
0011	3	3	-4
0100	4	4	-3
0101	5	5	-2
0110	6	6	-1
0111	7	7	0
1000	8	-8	1
1001	9	-7	2
1010	10	-6	3
1011	11	-5	4
1100	12	-4	5
1101	13	-3	6
1110	14	-2	7
1111	15	-1	8

To read more on this:
https://blog.angularindepth.com/the-mechanics-behind-exponent-bias-in-floating-point-9b3185083528

IEEE 32bit Floating-Point Number Representation

- Zero:
  - Exponent: All zeros
  - Fraction: All zeros
  - 0 has two different representations: +0 and -0.
- Infinity:
  - Exponent: All ones
  - Fraction: All zeros
- Not a number (NaN):
  - Exponent: All ones
  - Fraction: non-zero fraction.

Operations producing NaN:
wikipedia.org/wiki/NaN#Operations_generating_NaN

http://steve.hollasch.net/cgindex/coding/ieeefloat.html



IEEE 32bit Floating-Point Number Representation

- What is the maximum positive IEEE floating point value that can be stored?
  - Just less than 2^{128} [$(2 - 2^{-23}) \times 2^{127}$ to be specific]
  - Why? 2^{128} is reserved for NaN.

- Check out these useful links:
  - http://steve.hollasch.net/cgindex/coding/ieeefloat.html
  - http://babbage.cs.qc.cuny.edu/IEEE-754.old/Decimal.html

IEEE 32bit Floating-Point Number Representation

- Now consider 4.1:
  - 4 => $(100)_2$
  - 0.1 =>
    - x 2 = 0.2 = 0 + 0.2
    - x 2 = 0.4 = 0 + 0.4
    - x 2 = 0.8 = 0 + 0.8
    - x 2 = 1.6 = 1 + 0.6
    - x 2 = 1.2 = 1 + 0.2
    - x 2 = 0.4 = 0 + 0.4
    - x 2 = 0.8 = 0 + 0.8
    - …….

- So,
  - Representing a fraction which is a multiple of $1/2^n$ is lossless.
  - Representing a fraction which is not a multiple of $1/2^n$ leads to accuracy loss.



BINARY REPRESENTATION OF TEXT ETC.

Binary Representation of Textual Information

- Characters are mapped onto binary numbers
  - ASCII (American Standard Code for Information Interchange) code set
    - Originally: 7 bits per character; 128 character codes
  - Unicode code set
    - 16 bits per character
  - UTF-8 (Universal Character Set Transformation Format) code set.
    - Variable number of 8-bits.

METU Computer Engineering

ASCII
7 bits long

Dec	Bin	Char	Dec	Bin	Char	Dec	Bin	Char	Dec	Bin	Char
0	0000 0000	[NUL]	32	0010 0000	space	64	0100 0000	@	96	0110 0000	`
1	0000 0001	[SOH]	33	0010 0001	!	65	0100 0001	A	97	0110 0001	a
2	0000 0010	[STX]	34	0010 0010	"	66	0100 0010	B	98	0110 0010	b
3	0000 0011	[ETX]	35	0010 0011	#	67	0100 0011	C	99	0110 0011	c
4	0000 0100	[EOT]	36	0010 0100	$	68	0100 0100	D	100	0110 0100	d
5	0000 0101	[ENQ]	37	0010 0101	%	69	0100 0101	E	101	0110 0101	e
6	0000 0110	[ACK]	38	0010 0110	&	70	0100 0110	F	102	0110 0110	f
7	0000 0111	[BEL]	39	0010 0111	'	71	0100 0111	G	103	0110 0111	g
8	0000 1000	[BS]	40	0010 1000	(	72	0100 1000	H	104	0110 1000	h
9	0000 1001	[TAB]	41	0010 1001	)	73	0100 1001	I	105	0110 1001	i
10	0000 1010	[LF]	42	0010 1010	*	74	0100 1010	J	106	0110 1010	j
11	0000 1011	[VT]	43	0010 1011	+	75	0100 1011	K	107	0110 1011	k
12	0000 1100	[FF]	44	0010 1100	,	76	0100 1100	L	108	0110 1100	l
13	0000 1101	[CR]	45	0010 1101	-	77	0100 1101	M	109	0110 1101	m
14	0000 1110	[SO]	46	0010 1110	.	78	0100 1110	N	110	0110 1110	n
15	0000 1111	[SI]	47	0010 1111	/	79	0100 1111	O	111	0110 1111	o
16	0001 0000	[DLE]	48	0011 0000	0	80	0101 0000	P	112	0111 0000	p
17	0001 0001	[DC1]	49	0011 0001	1	81	0101 0001	Q	113	0111 0001	q
18	0001 0010	[DC2]	50	0011 0010	2	82	0101 0010	R	114	0111 0010	r
19	0001 0011	[DC3]	51	0011 0011	3	83	0101 0011	S	115	0111 0011	s
20	0001 0100	[DC4]	52	0011 0100	4	84	0101 0100	T	116	0111 0100	t
21	0001 0101	[NAK]	53	0011 0101	5	85	0101 0101	U	117	0111 0101	u
22	0001 0110	[SYN]	54	0011 0110	6	86	0101 0110	V	118	0111 0110	v
23	0001 0111	[ETB]	55	0011 0111	7	87	0101 0111	W	119	0111 0111	w
24	0001 1000	[CAN]	56	0011 1000	8	88	0101 1000	X	120	0111 1000	x
25	0001 1001	[EM]	57	0011 1001	9	89	0101 1001	Y	121	0111 1001	y
26	0001 1010	[SUB]	58	0011 1010	:	90	0101 1010	Z	122	0111 1010	z
27	0001 1011	[ESC]	59	0011 1011	;	91	0101 1011	[	123	0111 1011	{
28	0001 1100	[FS]	60	0011 1100	<	92	0101 1100	\	124	0111 1100	|
29	0001 1101	[GS]	61	0011 1101	=	93	0101 1101	]	125	0111 1101	}
30	0001 1110	[RS]	62	0011 1110	>	94	0101 1110	^	126	0111 1110	~
31	0001 1111	[US]	63	0011 1111	?	95	0101 1111	_	127	0111 1111	[DEL]

Some books make a mistake about the number of characters in the ASCII table.

To store the 4-character string "BAD!" in memory, the computer would store the binary representation of each individual character using the above 8-bit code.

BAD! = 00000010 00000001 00000100 10000001
B A D !

We have indicated above that the 8-bit numeric quantity 10000001 is interpreted as the character "!". However, as we mentioned earlier, the only way a computer knows that the 8-bit value 10000001 represents the symbol "!" and not the unsigned integer value 129 (128 + 1) or the signed integer value –1 (sign bit = negative, magnitude is 1) is by the context in which it is used. If these 8 bits are sent to a display device that expects to be given characters, then this value will be interpreted as an "!". If, on the other hand, this 8-bit value is sent to an arithmetic unit that adds unsigned numbers, then it will be interpreted as a 129 in order to make the addition operation meaningful.

To facilitate the exchange of textual information, such as word processing documents and electronic mail, between computer systems, it would be most helpful if everyone used the same code mapping. Fortunately, this is pretty much the case. Currently the most widely used code for representing characters internally in a computer system is called **ASCII**, an acronym for the *A*merican *S*tandard *C*ode for *I*nformation *I*nterchange. ASCII is an international standard for representing textual information in the majority of computers. It uses 8 bits to represent each character, so it is able to encode a total of $2^8 = 256$ different characters. These are assigned the integer values 0 to 255. However, only the numbers 32 to 126 have been assigned so far to printable characters. The remainder either are unassigned or are used for nonprinting control characters such as tab, form feed, and return. Figure 4.3 shows the ASCII conversion table for the numerical values 32–126.

However, a new code set called **UNICODE** is rapidly gaining popularity because it uses a 16-bit representation for characters rather than the 8-bit format of ASCII. This means that it is able to represent $2^{16} = 65{,}536$ unique characters instead of the $2^8 = 256$ of ASCII. It may seem like 256 characters are more than enough to represent all the textual symbols that we would ever need—for example, 26 uppercase letters, 26 lowercase letters, 10 digits, and a few dozen special symbols, such as +=–{}][\;'"?><.,;'%$#@. Add that all together and it still totals only about 100 symbols, far less than the 256 that can be represented in ASCII. However, that is true only if we limit our work to Arabic numerals and the Roman alphabet. The world grows more connected all the time—helped along by computers, networks, and the Web—and it is critically important that computers represent and exchange textual information using alphabets in addition to these 26 letters and 10 digits. When we start assigning codes to symbols drawn from alphabets such as Russian, Arabic, Chinese, Hebrew, Greek, Thai, Bengali, and Braille, as well as mathematical symbols and special linguistic marks such as tilde, umlaut, and accent grave, it becomes clear that ASCII does not have nearly enough room to represent them all. However, UNICODE, with space for over 65,000 symbols, is large enough to accommodate all these symbols and many more to come. In fact, UNICODE has defined standard code mappings for more

UTF-8 Illustrated

METU Computer Engineering

Bits	Last code point	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
7	U+007F	0xxxxxxx					
11	U+07FF	110xxxxx	10xxxxxx				
16	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx			
21	U+1FFFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx		
26	U+3FFFFFF	111110xx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	
31	U+7FFFFFFF	1111110x	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx

Character		Binary code	Binary UTF-8
$	U+0024	0100100	00100100
¢	U+00A2	00010100010	11000010 10100010
€	U+20AC	0010000010101100	11100010 10000010 10101100
𤭢	U+24B62	000100100101101100010	11110000 10100100 10101101 10100010

How about a text?

- Text in a computer has two alternative representations:

1. A fixed-length number representing the length of the text followed by the binary values of the characters in the text.
   - Ex: “ABC” =>

   00000011 01000001 01000001 01000001 (3 ‘A’ ‘B’ ‘C’)

2. Binary values of the characters in the text ended with a unique marker, like “00000000” which has no value in the ASCII table.
   - Ex: “ABC” =>

   01000001 01000001 01000001 00000000 (‘A’ ‘B’ ‘C’ END)

METU Computer Engineering



Binary Representation of Sound and Images

- Multimedia data is sampled to store a digital form, with or without detectable differences
- Representing sound data
  - Sound data must be digitized for storage in a computer
  - Digitizing means periodic sampling of amplitude values

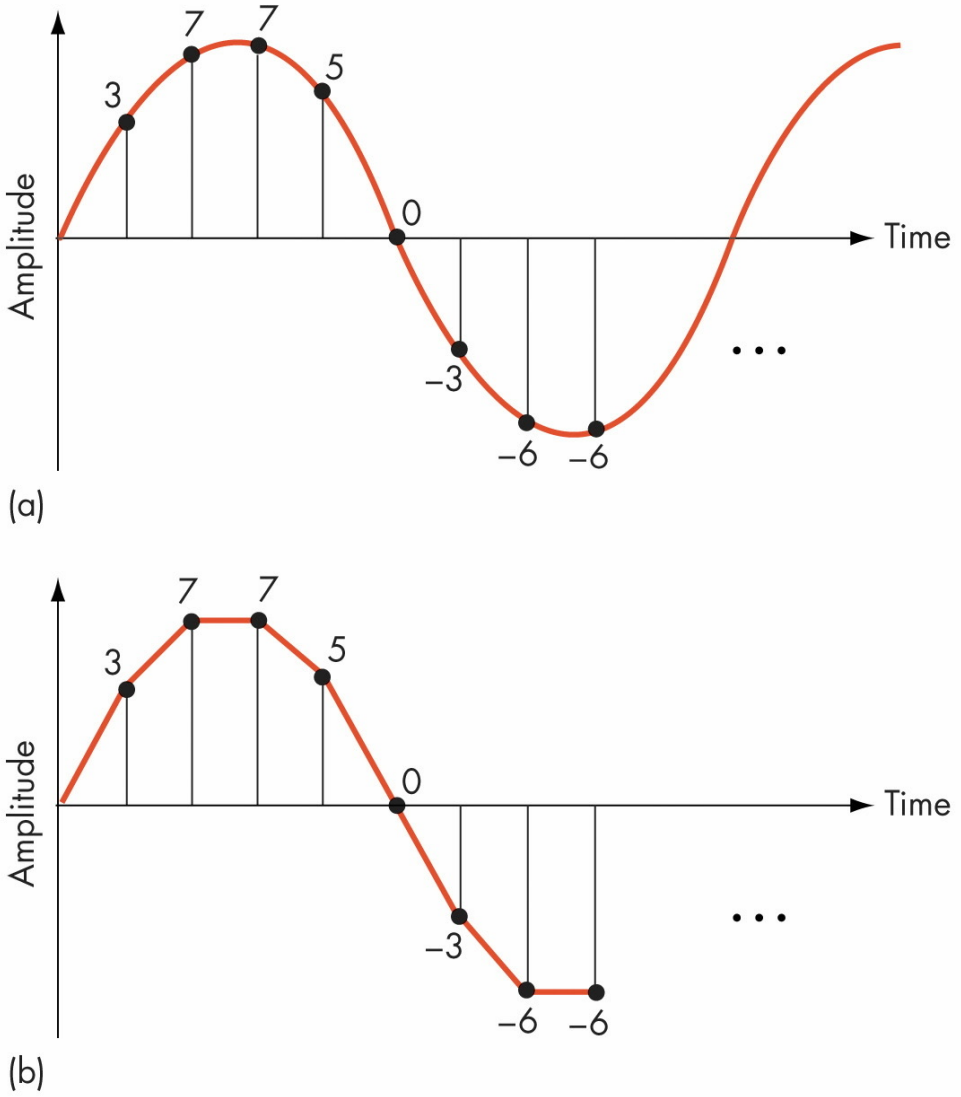
Binary Representation of Sound and Images (continued)

- From samples, original sound may be approximated
- To improve the approximation:
  - Sample more frequently (*increase sampling rate)*
  - Use more bits for each sample value *(↑ bit depth)*

Digitization of an Analog Signal

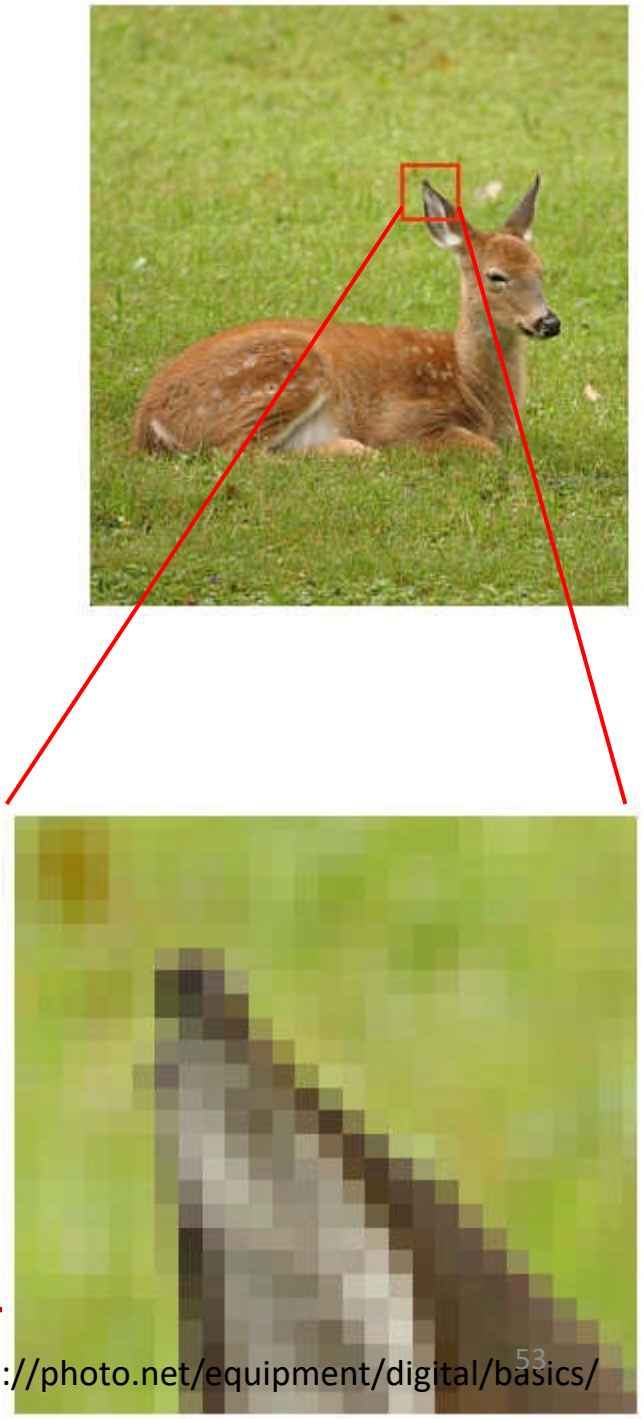
(a) Sampling the Original Signal

(b) Recreating the Signal from the Sampled Values



Binary Representation of Sound and Images (continued)

- Representing image data
  - Images are sampled by reading color and intensity values at even intervals across the image
  - Each sampled point is a pixel
  - Image quality depends on number of bits at each pixel
  - More image information: http://cat.xula.edu/tutorials/imaging/grayscale.php



http://photo.net/equipment/digital/basics/



The Reliability of Binary Representation

- Electronic devices are most reliable in a bistable environment
- Bistable environment
  - Distinguishing only two electronic states
    - Current flowing or not, or
    - Direction of flow
- Computers are bistable: hence binary representations
- See the following video for contrasting the design of a binary computer and a ternary one:

  https://www.youtube.com/watch?v=fXwSFhUVFmE